



UNIVERSIDADE ESTADUAL DE SANTA CRUZ - UESC
CURSO SUPERIOR DE GRADUAÇÃO
EM CIÊNCIA DA COMPUTAÇÃO

**BENCHMARK DE GERAÇÃO DO CONJUNTO DE MANDELBROT (ESCAPE-TIME):
IMPLEMENTAÇÃO SEQUENCIAL E PARALELA EM MEMÓRIA COMPARTILHADA COM
OPENMP**

ILHÉUS - BA

22 de setembro de 2026

**ALÍCIA OLIVEIRA ARAÚJO
ISABELA MADUREIRA ARGOLO**

**BENCHMARK DE GERAÇÃO DO CONJUNTO DE MANDELBROT (ESCAPE-TIME):
IMPLEMENTAÇÃO SEQUENCIAL E PARALELA EM MEMÓRIA COMPARTILHADA COM
OPENMP**

Relatório apresentado como requisito parcial à disciplina de DEC107 - Processamento Paralelo, da Universidade Estadual de Santa Cruz, pelo curso de Ciência da Computação, turma de 2026.2.

Orientador (a): Prof. Dr. Esbel Tomás Valero Orellana.

ILHÉUS - BA

22 de setembro de 2026

SUMÁRIO

1. INTRODUÇÃO.....	4
2. METODOLOGIA.....	4
2.1. Implementação Sequencial.....	4
2.2. Implementação Paralela com OpenMP.....	5
2.3. Ambiente e Procedimento Experimental.....	5
2.4. Métricas.....	6
3. RESULTADOS.....	7
3.1. Execução Sequencial.....	7
3.2. Escalabilidade Forte.....	7
3.3. Políticas de Escalonamento.....	9
3.4. Balanceamento de Carga.....	11
3.5. Escalabilidade Fraca.....	13
4. DISCUSSÃO.....	14
5. CONCLUSÃO.....	15
6. REFERÊNCIAS.....	16

1. INTRODUÇÃO

O conjunto de Mandelbrot pode ser representado pelo método *escape-time*, em que cada pixel da imagem corresponde a um ponto c do plano complexo. Para cada ponto, inicia-se $z_0=0$ e aplica-se repetidamente a expressão

$$z_{n+1} = z_n^2 + c$$

até que $|z|^2 > 4$ ou seja atingido o número máximo de iterações. A quantidade de iterações realizada para cada pixel é utilizada na geração da imagem e também representa o custo computacional daquele ponto.

O cálculo de cada pixel é independente dos demais, o que permite distribuir o trabalho entre diferentes threads. Porém, o custo dos pixels não é uniforme. Alguns pontos escapam rapidamente, enquanto outros, principalmente aqueles dentro do conjunto ou próximos da fronteira, podem atingir o número máximo de iterações. Essa diferença torna o balanceamento de carga um ponto importante na paralelização do problema.

Nesta etapa, a implementação sequencial em C foi utilizada como base para uma versão paralela com OpenMP. Foram analisados o número de threads, as políticas de escalonamento *static*, *dynamic* e *guided*, diferentes tamanhos de *chunk* e o comportamento da aplicação em testes de escalabilidade forte e fraca.

O caso padrão utiliza resolução de 4096×4096 , região real $[-2, 0; 1, 0]$, região imaginária $[-1, 5; 1, 5]$ e $MAX_ITER = 1000$. Para a análise de desbalanceamento também foi utilizado o Vale dos Cavalos-Marinheiros, com centro em $(-0,743643887; 0,131825904)$, largura de $3,0 \times 10^{-3}$ e $MAX_ITER = 5000$.

2. METODOLOGIA

2.1. Implementação Sequencial

A versão sequencial foi desenvolvida em C, com base no código disponibilizado pelo professor em Python, e utilizada como referência para a implementação paralela.

A função mandelbrot recebe os limites da região do plano complexo, a largura e a altura da imagem e o número máximo de iterações. Para armazenar os resultados, é criada uma matriz do tipo *int32_t*.

A imagem é percorrida por linhas e colunas. Para cada pixel são calculadas suas coordenadas no plano complexo e, a partir de $z = 0$, é aplicada a recorrência do Mandelbrot até que o ponto escape ou seja atingido *MAX_ITER*.

O critério de escape utilizado é $zr^2 + zi^2 > 4$.

As contagens são armazenadas em ordem *row-major*. A implementação gera quatro representações: a vista completa, o Vale dos Cavalos-Marinheiros, o mapa de custo computacional e uma região com formação em espiral.

2.2. Implementação Paralela com OpenMP

A versão paralela manteve o mesmo cálculo realizado para cada pixel. A principal alteração foi aplicada ao laço externo da função *mandelbrot*, responsável pelas linhas da imagem.

Foi utilizada a diretiva *parallel for* com *schedule(runtime)*, permitindo alterar a política de escalonamento por meio de *OMP_SCHEDULE* sem modificar o código. A quantidade de threads também pode ser definida utilizando *OMP_NUM_THREADS*.

Cada thread processa linhas diferentes e escreve em posições distintas da matriz *count*. Assim, não foi necessário utilizar regiões *critical* ou operações *atomic*.

Foram testadas as políticas *static*, *dynamic* e *guided*, com *chunks* de 1, 8 e 32 linhas. Para a medição de balanceamento foi utilizada uma versão instrumentada do programa, que registra o menor tempo, o maior tempo e o tempo médio entre as threads e calcula o fator de balanceamento de carga.

A versão paralela também permite informar a resolução pela linha de comando. Caso nenhum valor seja informado, são utilizados 4096×4096 pixels. Essa opção foi utilizada nos testes de escalabilidade fraca.

2.3. Ambiente e Procedimento Experimental

As configurações da máquina utilizada para a realização dos testes são:

Tabela 1 - Configurações do ambiente utilizado nos testes

Componente	Configuração
Processador	AMD Ryzen 5 3600X 6-Core Processor
Núcleos Físicos	6
Processadores Lógicos	12
Sistema Operacional	Windows 11 Pro, 25H2, 64 bits — build 26200.9457
Compilador	GCC 6.3.0 — MinGW.org GCC-6.3.0-1
Versão do OpenMP	OpenMP 4.5 (_OPENMP = 201511), runtime libgomp

Cada configuração foi executada três vezes e foi utilizada a mediana dos tempos. Os tempos de escrita dos arquivos foram desconsiderados na análise de desempenho, e os tempos da versão instrumentada foram utilizados apenas para o estudo do balanceamento de carga.

Na escalabilidade forte, foi mantida a resolução de 4096×4096 e utilizadas 1, 2, 4, 6 e 12 threads com *dynamic*, 1. Para a comparação das políticas foram utilizadas 12 threads, combinando *static*, *dynamic* e *guided* com *chunks* 1, 8 e 32.

Na escalabilidade fraca, o tamanho da imagem foi aumentado de acordo com o número de threads, buscando manter aproximadamente constante a quantidade de pixels por thread.

2.4. Métricas

O *speedup* foi calculado por:

$$S = \frac{T_{seq}}{T_p}$$

onde T_{seq} representa o tempo sequencial e T_p o tempo paralelo.

A eficiência é calculada por:

$$E = \frac{S_p}{p} \times 100$$

Para avaliar o balanceamento de carga, foi utilizado:

$$FLB = \frac{T_{max} - T_{min}}{T_{max}}$$

Quanto menor o FLB, menor a diferença entre os tempos de término das threads.

3. RESULTADOS

3.1. Execução Sequencial

Os tempos da versão sequencial são apresentados na tabela a seguir:

Tabela 2 - Tempos de versão sequencial

Região	Rep. 1 (s)	Rep. 2 (s)	Rep. 3 (s)	Mediana (s)
Vista Completa	6,774	6,810	6,792	6,792
Vale dos Cavalos-Marinheiros	3,008	3,012	3,013	3,012
Mapa de Custo	6,775	6,783	6,772	6,775
Zoom Espiral	63,176	63,235	63,158	63,176

Esses valores foram utilizados como referência para os cálculos de *speedup*.

3.2. Escalabilidade Forte

A tabela a seguir apresenta os resultados da Vista Completa utilizando *dynamic*, 1.

Tabela 3 - Escalabilidade forte na Vista Completa

Threads	Tempo Mediano (s)	Speedup	Eficiência
1	6,840	0,993	99,30%
2	3,483	1,950	97,50%
4	1,749	3,883	97,08%
6	1,196	5,679	94,65%
12	0,715	9,499	79,16%

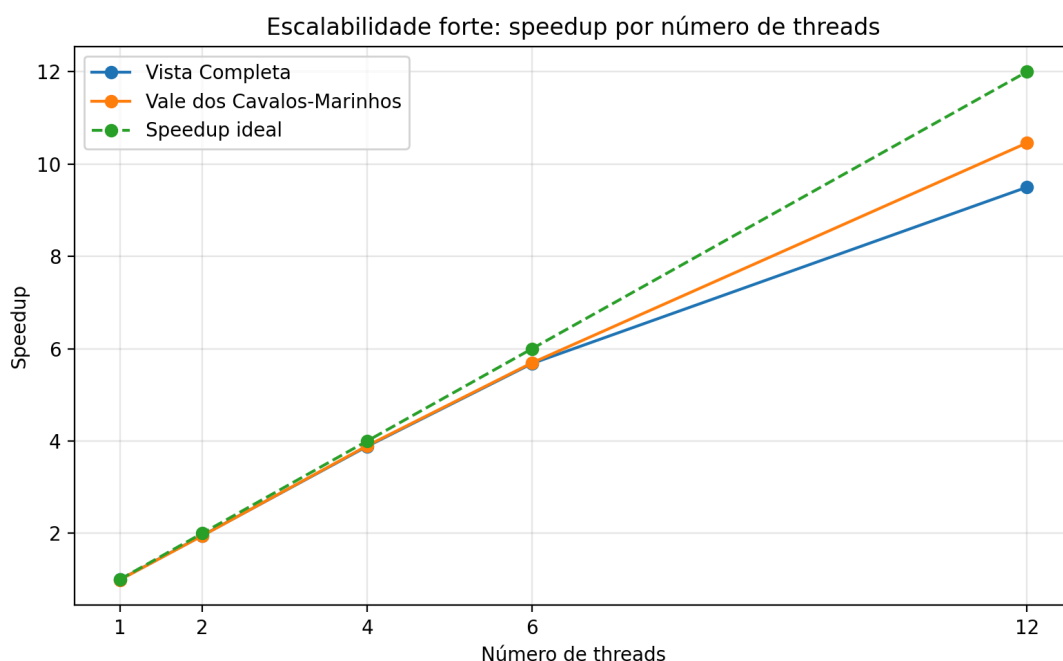
No Vale dos Cavalos-Marinheiros, com 12 threads, o tempo mediano foi de 0,288 s, com *speedup* de 10,458 e eficiência de 87,15%.

Os resultados mostram que o aumento do número de threads reduziu de forma consistente o tempo de execução. Até quatro threads, o *speedup* ficou bastante próximo do crescimento ideal, com eficiências superiores a 97% tanto na Vista Completa quanto no Vale. Com seis threads, a eficiência ainda permaneceu próxima de 95%, indicando bom aproveitamento dos seis núcleos físicos disponíveis no processador utilizado nos testes.

Com 12 threads, o tempo continuou diminuindo, mas o ganho deixou de crescer na mesma proporção. Na Vista Completa, o tempo passou de 1,196 s com seis threads para 0,715 s com 12, enquanto no Vale passou de 0,529 s para 0,288 s. Assim, mesmo utilizando os 12 processadores lógicos, ainda houve ganho de desempenho, porém acompanhado pela redução da eficiência para 79,16% e 87,15%, respectivamente.

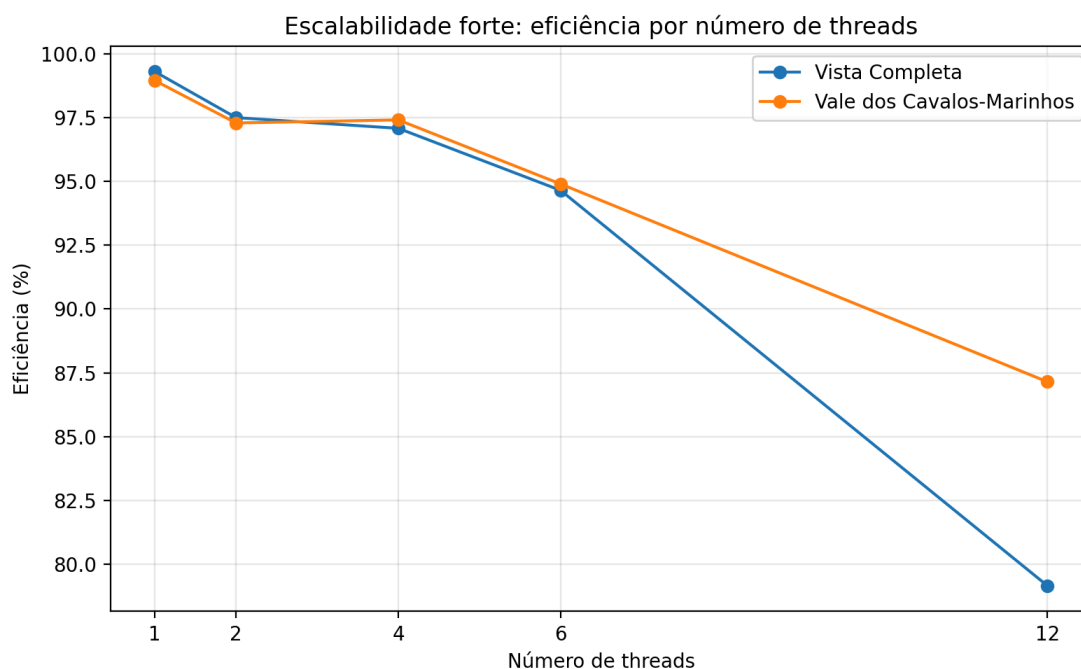
A execução OpenMP com apenas uma thread apresentou desempenho muito próximo da versão sequencial, mas ligeiramente inferior. Essa diferença pequena pode ser associada ao custo adicional necessário para criar e gerenciar a região paralela.

Figura 1 - Speedup na escalabilidade forte para a Vista Completa e o Vale dos Cavalos-Marinheiros em comparação com o crescimento ideal



A figura 1 permite visualizar o afastamento gradual em relação ao *speedup* ideal. Esse afastamento se torna mais evidente com 12 threads, principalmente na Vista Completa.

Figura 2 - Eficiência da implementação paralela na escalabilidade forte para a Vista Completa e o Vale dos Cavalos-Marinheiros



A redução da eficiência apresentada na Figura 2 ocorre principalmente na passagem de seis para 12 threads. Como o computador utilizado possui seis núcleos físicos e 12 processadores lógicos, os resultados indicam que o uso das threads adicionais ainda reduz o tempo, mas não duplica o desempenho.

3.3. Políticas de Escalonamento

Os testes das políticas foram realizados com 12 threads e resolução de 4096×4096 .

Tabela 4 - Comparação das políticas

Política	Chunk	Vista Completa (s)	Vale (s)
static	1	0,728	0,294
static	8	0,732	0,296
static	32	0,759	0,305
dynamic	1	0,703	0,291
dynamic	8	0,714	0,294
dynamic	32	0,713	0,306
guided	1	1,059	0,335

guided	8	1,050	0,334
guided	32	1,034	0,337

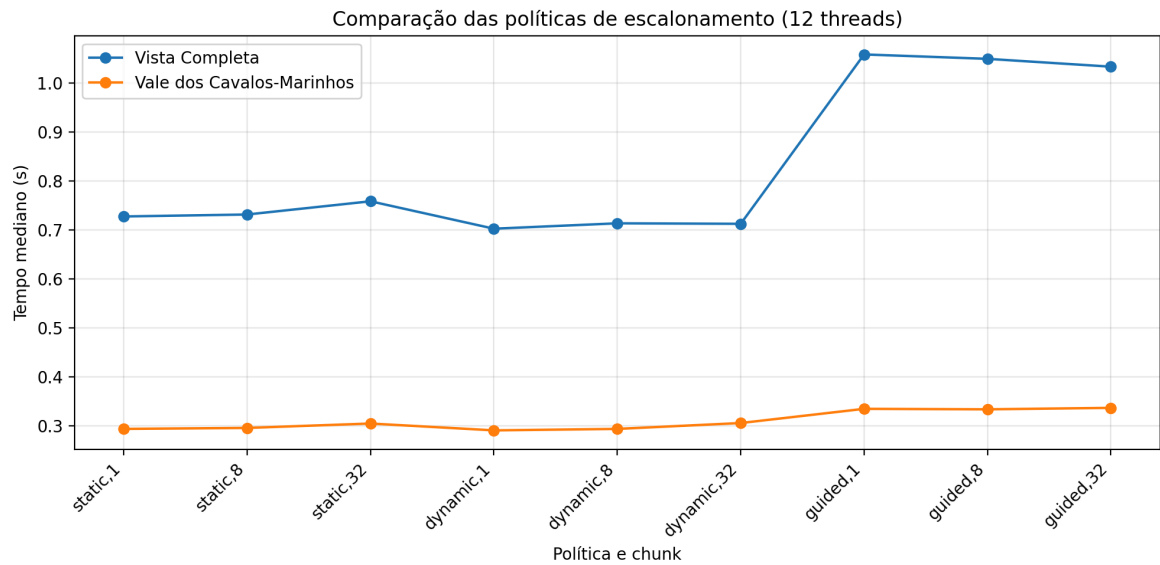
A configuração *dynamic*, 1 apresentou os menores tempos na Vista Completa, no Vale dos Cavalos-Marinheiros e no Zoom Espiral. No Mapa de Custo, o menor tempo foi obtido com *dynamic*, 8, porém a diferença em relação a *dynamic*, 1 foi de apenas 0,002s. Dessa forma, *dynamic*, 1 apresentou o comportamento mais consistente entre as regiões analisadas.

As políticas *static* e *dynamic* apresentaram resultados relativamente próximos quando foram utilizados *chunks* pequenos. Na Vista Completa, por exemplo, *static*, 1 levou 0,728s e *dynamic*, 1 levou 0,703s. No Vale, os tempos foram 0,294s e 0,291s. Isso mostra que, apesar da irregularidade do problema, a diferença entre essas duas políticas não foi grande nessas configurações.

O aumento do *chunk* para 32 não trouxe melhoria de desempenho no Vale. Em *static*, o tempo aumentou de 0,294s para 0,305s e, em *dynamic*, de 0,291s para 0,306s. Isso sugere que blocos maiores diminuíram a flexibilidade para redistribuir o trabalho entre as threads.

Já *guided* apresentou os maiores tempos nas regiões avaliadas. Na Vista Completa, sua melhor configuração foi *guided*, 32, com 1,034s, enquanto *dynamic*, 1 apresentou 0,703s. No Vale, a diferença foi menor, mas *guided* também ficou acima das demais políticas.

Figura 3 - Comparação dos tempos medianos das políticas de escalonamento e tamanhos de chunk utilizando 12 threads



A Figura 3 facilita a comparação entre as configurações e evidencia principalmente a diferença de *guided* em relação a *static* e *dynamic* na Vista Completa.

3.4. Balanceamento de Carga

O Vale dos Cavalos-Marinheiros foi utilizado como principal caso para a análise do balanceamento.

Tabela 5 - Balanceamento no Vale dos Cavalos-Marinheiros

Política	Chunk	Tmin (s)	Tmax (s)	Tmédio (s)	FLB
static	1	0,284	0,296	0,287750	0,040541
static	8	0,279	0,296	0,288917	0,057432
static	32	0,249	0,313	0,288083	0,204473
dynamic	1	0,291	0,293	0,292000	0,006826
dynamic	8	0,284	0,294	0,288083	0,034014
dynamic	32	0,266	0,309	0,289250	0,139159
guided	1	0,280	0,343	0,288333	0,189504
guided	8	0,277	0,339	0,286917	0,183432
guided	32	0,266	0,336	0,286583	0,217262

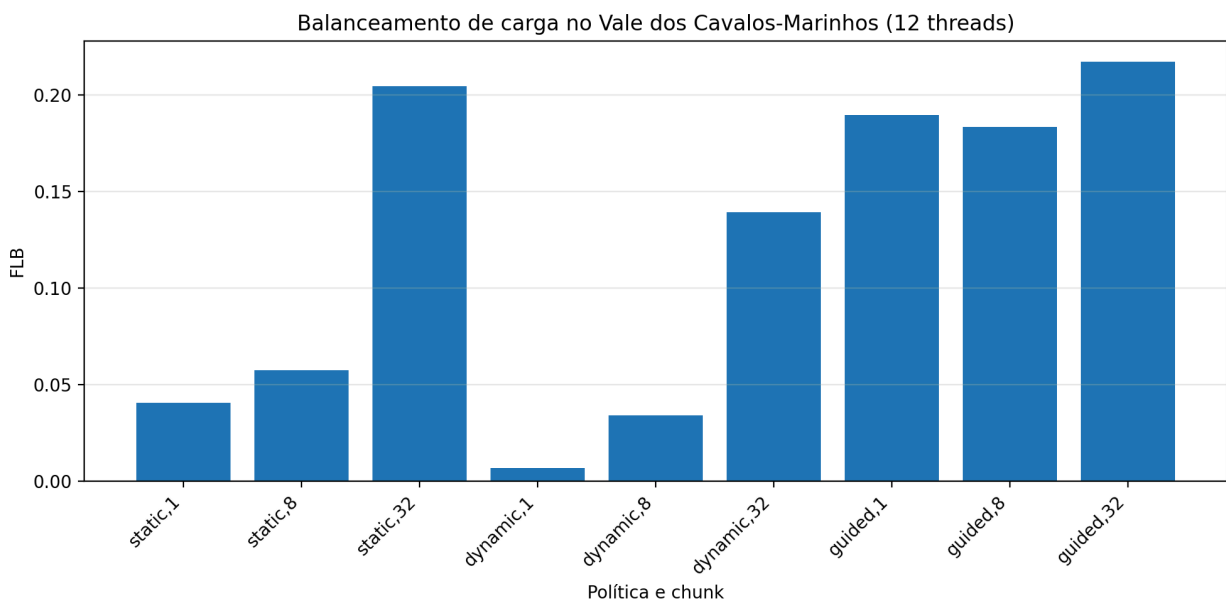
Dynamic, 1 apresentou o menor FLB, com 0,006826. Nessa configuração, o menor e o maior tempo registrados entre as threads foram 0,291s e 0,293s, mostrando uma diferença muito pequena no término do trabalho.

Dynamic, 8 também apresentou bom balanceamento, com FLB de 0,034014. Já o aumento do *chunk* para 32 piorou esse comportamento: o FLB chegou a 0,139159 em *dynamic, 32* e 0,204473 em *static, 32*.

As configurações *guided* apresentaram os maiores níveis de desbalanceamento no Vale. O FLB variou entre 0,183432 e 0,217262, mostrando uma diferença maior entre as threads que terminaram primeiro e aquelas que permaneceram executando por mais tempo.

Esses resultados complementam a análise de tempo da seção anterior. *Dynamic, 1* não apenas apresentou o menor tempo no Vale, como também apresentou a distribuição mais equilibrada entre as threads. Portanto, nesse caso, o menor tempo de execução e o melhor balanceamento ocorreram na mesma configuração.

Figura 4 - Fator de balanceamento de carga no Vale dos Cavalos-Marinheiros para as políticas e tamanhos de chunk testados



Na Figura 4, valores menores representam maior equilíbrio. É possível observar a diferença entre *dynamic, 1* e as configurações com *chunk 32*, além dos valores mais elevados obtidos por *guided*.

3.5. Escalabilidade Fraca

Na escalabilidade fraca, foram utilizadas resoluções maiores conforme o número de threads aumentou.

Tabela 6 - Escalabilidade fraca

Threads	Resolução	Vista Completa (s)	Vale (s)
1	1024 × 1024	0,440	0,194
2	1448 × 1448	0,435	0,196
4	2048 × 2048	0,436	0,197
6	2508 × 2508	0,444	0,200
12	3547 × 3547	0,549	0,220

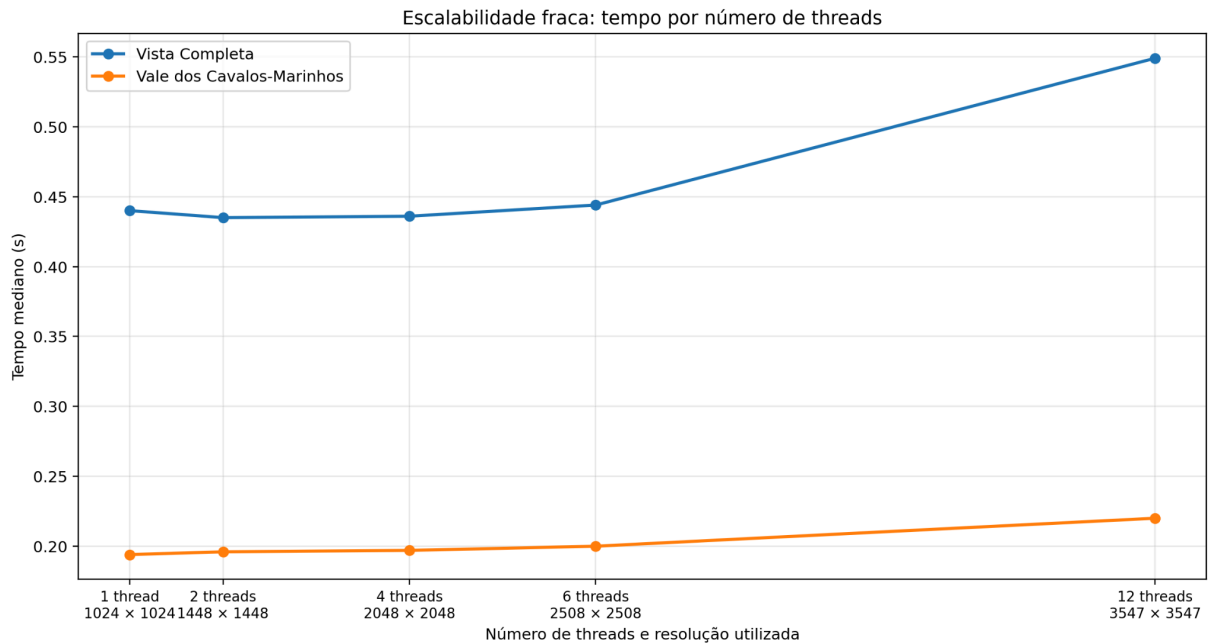
Entre uma e seis threads, os tempos permaneceram bastante próximos, mesmo com o aumento da resolução. Na Vista Completa, o tempo variou entre 0,435s e 0,444s, enquanto no Vale ficou entre 0,194s e 0,200s. Esse comportamento mostra que, nesse intervalo, o aumento dos recursos conseguiu compensar de forma adequada o crescimento do tamanho do problema.

Com 12 threads, essa tendência mudou. O tempo da Vista Completa chegou a 0,549s, representando aumento de 24,8% em relação à execução com uma thread. No Vale, o tempo passou de 0,194 s para 0,220s, aumento de 13,4%. O Mapa de Custo e o Zoom Espiral também apresentaram aumentos de 10,3% e 10,6%, respectivamente.

Assim, a escalabilidade fraca foi bastante próxima do comportamento esperado até seis threads. Na configuração com 12 threads, o crescimento dos recursos não foi suficiente para manter o mesmo tempo de execução.

Como a máquina possui seis núcleos físicos e 12 processadores lógicos, esse resultado também coincide com o ponto em que passam a ser utilizadas duas threads lógicas por núcleo. Esse fator pode contribuir para a perda observada, embora os testes realizados não permitam separar isoladamente o efeito de cada recurso de hardware.

Figura 5 - Tempos medianos da escalabilidade fraca para a Vista Completa e o Vale dos Cavalos-Marinheiros em diferentes números de threads e resoluções



A Figura 5 evidencia a estabilidade dos tempos até seis threads e o aumento observado na configuração com 12 threads.

4. DISCUSSÃO

Os resultados mostram que a paralelização com OpenMP foi eficiente para o cálculo do conjunto de Mandelbrot. Na escalabilidade forte, o desempenho ficou próximo do ideal até seis threads, com redução consistente do tempo de execução. Com 12 threads, o tempo continuou diminuindo, mas a eficiência caiu, indicando que o aumento dos recursos já não gerava ganho na mesma proporção.

Na comparação entre as políticas de escalonamento, *dynamic*, 1 apresentou o comportamento mais consistente. Além de obter os menores tempos em três das quatro regiões, também apresentou o melhor balanceamento no Vale dos Cavalos-Marinheiros. Isso indica que a distribuição dinâmica com blocos pequenos conseguiu lidar melhor com a diferença de custo entre as linhas da imagem.

O tamanho do *chunk* também teve influência. Em *static* e *dynamic*, o uso de *chunk* 32 aumentou o desbalanceamento, enquanto valores menores apresentaram resultados mais equilibrados. A política *guided*, por sua vez, apresentou tempos maiores e pior balanceamento nas configurações analisadas.

Na escalabilidade fraca, os tempos permaneceram praticamente constantes até seis threads, mesmo com o aumento da resolução. Com 12 threads, houve aumento do tempo nas regiões avaliadas, mostrando uma perda de eficiência na maior configuração testada.

De forma geral, os testes mostram que o desempenho não depende apenas da quantidade de threads, mas também da estratégia usada para distribuir o trabalho. Entre as configurações avaliadas, *dynamic, 1* apresentou o melhor equilíbrio entre tempo de execução e balanceamento de carga.

5. CONCLUSÃO

A implementação com OpenMP permitiu reduzir o tempo de execução do conjunto de Mandelbrot mantendo a mesma organização do cálculo utilizado na versão sequencial.

Na escalabilidade forte, a aplicação apresentou comportamento próximo do ideal até quatro threads e manteve boa eficiência com seis. Com 12 threads foram obtidos *speedups* de 9,499 na Vista Completa e 10,458 no Vale dos Cavalos-Marinheiros, embora com redução da eficiência.

Entre as políticas de escalonamento avaliadas, *dynamic, 1* apresentou o conjunto mais consistente de resultados. Essa configuração obteve os menores tempos em três das quatro regiões e também o menor fator de balanceamento no Vale, com FLB de 0,006826.

A escalabilidade fraca apresentou tempos praticamente constantes até seis threads. Com 12 threads, houve aumento do tempo nas quatro regiões, mostrando uma perda de eficiência quando foi utilizada a maior configuração testada.

De forma geral, os experimentos mostraram que o desempenho do Mandelbrot depende não apenas da quantidade de threads, mas também da forma como o trabalho é distribuído. A escolha da política e do tamanho do *chunk* teve influência tanto no tempo de execução quanto no balanceamento de carga.

6. REFERÊNCIAS

OPENMP ARCHITECTURE REVIEW BOARD. **OpenMP Application Programming Interface: version 5.1**. 2020. Disponível em: <https://www.openmp.org/spec-html/5.1/openmp.html>. Acesso em: 18 set. 2026.

ORELLANA, Esbel T. Valero. **DEC107 Processamento Paralelo: Aula 03 – Caracterização de desempenho em programas paralelos**. Ilhéus: Universidade Estadual de Santa Cruz, 2026.

ORELLANA, Esbel T. Valero. **DEC107 Processamento Paralelo: Aula 04 – Arquiteturas paralelas e introdução a OpenMP**. Ilhéus: Universidade Estadual de Santa Cruz, 2026.

ORELLANA, Esbel T. Valero. **CET107 Processamento Paralelo: Aulas 06, 07 e 08 – Diretivas OpenMP; Aula 09 – API OpenMP**. Ilhéus: Universidade Estadual de Santa Cruz, 2026.

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Nós, **Alícia Oliveira Araújo** e **Isabela Madureira Argolo**, regularmente matriculadas no curso de Ciência da Computação da Universidade Estadual de Santa Cruz, declaramos, para os devidos fins, que, na elaboração do trabalho intitulado “**Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação Sequencial e Paralela em Memória Compartilhada com OpenMP**”, submetido como requisito para a disciplina **DEC107 — Processamento Paralelo**, utilizamos ferramentas de Inteligência Artificial generativa como recursos de apoio.

Ferramenta utilizada: ChatGPT.

Detalhamento do uso: a ferramenta foi utilizada como apoio nas seguintes etapas do projeto:

- auxílio na revisão do código sequencial e paralelo;
- apoio na análise de possíveis ajustes na implementação com OpenMP;
- auxílio na estruturação dos procedimentos de teste;
- geração de gráficos a partir dos dados coletados;
- revisão da organização, clareza e escrita do relatório.

Todo o código utilizado no projeto, os procedimentos de teste, os resultados obtidos e os textos produzidos com auxílio da ferramenta foram revisados e validados pelas autoras.

Termo de Responsabilidade: Declaramos estar cientes de que ferramentas de Inteligência Artificial podem apresentar erros, inconsistências ou informações incorretas. Dessa forma, assumimos integral responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido. Atestamos que o código-fonte, os resultados dos testes, as análises e os textos desenvolvidos com auxílio de IA foram revisados, testados e validados por nós, de forma a garantir a compreensão do trabalho realizado e a integridade acadêmica do material entregue.

Ilhéus – BA, 22 de setembro de 2026.

Alícia Oliveira Araújo

Matrícula: 202310348

Isabela Madureira Argolo

Matrícula: 202310353